

KARADENİZ TEKNİK ÜNİVERSİTESİ — BİLGİSAYAR BİLİMLERİ

Hafta 4 Quiz

Kaynak Yönetimi ve Akıllı İşaretçiler

Doç. Dr. Tolga Berber · https://avesis.ktu.edu.tr/tberber

Öğrenme fazları: 1 • Ön-öğretim 2 • İçerik 3 • Katılım 4 • Değerlendirme 5 • İzleme

Soru 1 (10 puan)

4 • Değerlendirme Hedef 1

`araclar::IntTampon` sınıfı bir `int* veri` ve `int boyut` üyesi tutar; kurucusu `new[]` ile bellek ayırır, yıkıcısı `delete[]` ile bırakır. Sınıf kendi kopya kurucusunu YAZMAZSA, derleyicinin ürettiği varsayılan kopya kurucu neden tehlikelidir?

(a) Derleyici varsayılan kopya kurucu üretmez; `IntTampon b = a;` gibi bir satır derlenmez.

(b) Varsayılan kopya kurucu üye üye (memberwise) kopyalar; `veri` işaretçisi için bu yalnızca adresin kopyalanması demektir — iki nesne aynı belleği paylaşır ve ikisi de kapsam dışına çıkınca aynı bellek iki kez `delete[]` edilir.

(c) Varsayılan kopya kurucu `veri` dizisinin içeriğini otomatik olarak derin kopyalar, bu yüzden herhangi bir risk yoktur.

(d) Varsayılan kopya kurucu yalnızca `boyut` üyesini kopyalar, `veri` üyesini `nullptr` yapar.

Soru 2 (10 puan)

4 • Değerlendirme Hedef 2

Rule of 3 ilkesi ne söyler?

(a) Bir sınıfta en fazla 3 üye fonksiyon bulunmalıdır.

(b) Bir sınıf yıkıcı, kopya kurucu veya kopya atama operatöründen birini elle yazmak zorundaysa, genelde üçünü de elle yazmak zorundadır — üçü de aynı soruyu (bu sınıf bir kaynağın sahibi mi?) cevaplar.

(c) Bir sınıfın kurucusu en fazla 3 parametre alabilir.

(d) Bir sınıf, kaynak yönetiyorsa yalnızca `unique_ptr` kullanmalı, asla ham işaretçi tutmamalıdır.

Soru 3 (10 puan)

4 • Değerlendirme Hedef 2

Aşağıdaki sınıf tanımını inceleyiniz:

```
class TekilKart {
public:
   explicit TekilKart(int numara) : numara_(numara) {}
   TekilKart(const TekilKart&) = delete;
   TekilKart& operator=(const TekilKart&) = delete;
   int numara() const { return numara_; }
```

Soru 3 (devamı)

```
private:
   int numara_;
};

TekilKart k1(42);
TekilKart k2 = k1; // (*)
```

(*) ile işaretli satır için hangisi doğrudur?

(a) Satır normal şekilde derlenir; `k2`, `k1`'in üye üye kopyasını alır.

(b) Satır derlenmez: kopya kurucu `= delete` ile açıkça yasaklandığından, derleyici bu çağrıyı bir hata olarak reddeder.

(c) Satır derlenir ama çalışma zamanında (runtime) bir istisna fırlatır.

(d) Satır yalnızca `TekilKart` sınıfı `= default` kopya kurucusuna sahip olsaydı derlenmezdi; burada sorun yoktur.

Soru 4 (10 puan)

4 • Değerlendirme Hedef 3

`unique_ptr` ile `shared_ptr` arasındaki temel fark ve tercih kuralı için hangisi DOĞRUDUR?

(a) İkisi de aynı kaynağı aynı anda paylaşabilir; aralarındaki tek fark isimleridir.

(b) `unique_ptr` tekil ve kopyalanamaz (yalnız taşınabilir) bir sahiplik sağlar; `shared_ptr` referans sayımıyla paylaşılan sahiplik sağlar. Varsayılan tercih daha ucuz/açık olan `unique_ptr`'dır; gerçekten birden çok sahip aynı anda kaynağı yaşatmalıysa `shared_ptr` kullanılır.

(c) `shared_ptr` her zaman `unique_ptr`'dan daha verimlidir, bu yüzden her durumda tercih edilmelidir.

(d) `unique_ptr` yalnızca diziler için, `shared_ptr` yalnızca tekil nesneler için kullanılabilir.

Soru 5 (15 puan)

4 • Değerlendirme Hedef 1

`IntTampon`'un kopya kurucusu ve kopya atama operatörü derin kopya yapacak şekilde yazılmıştır. Aşağıdaki `main` kesitini inceleyiniz:

```
araclar::IntTampon a(3);
for (int i = 0; i < a.boyut; ++i) a.veri[i] = i + 1; // a = {1,2,3}

araclar::IntTampon b = a; // kopya kurucu (derin kopya)
a.veri[0] = 99; // yalniz a degisir
std::cout << "a.veri[0]=" << a.veri[0]
          << " b.veri[0]=" << b.veri[0] << "\n";

araclar::IntTampon c(3);
c = a; // kopya atama (derin kopya)
a.veri[1] = 42; // yalniz a tekrar degisir
std::cout << "a.veri[1]=" << a.veri[1]
          << " c.veri[1]=" << c.veri[1] << "\n";
```

Bu iki `std::cout` satırının konsola bastığı çıktıyı yazınız.

Soru 6 (10 puan)

4 • Değerlendirme Hedef 3

`AkilliTampon` sınıfı, `std::unique_ptr<int[]>` tipli tek bir `veri_` üyesiyle kuruludur (Rule of 0 — kullanıcı tanımlı yıkıcı/kopya kurucu/kopya atama YOKTUR). Aşağıdaki kesiti inceleyiniz:

```
araclar::AkilliTampon a(5);
for (int i = 0; i < a.boyut(); ++i) a[i] = i * 10;

araclar::AkilliTampon b = std::move(a); // sahiplik b'ye tasinir

araclar::AkilliTampon c = b; // (*)
```

(*) ile işaretli satır derlenir mi? Nedenini kısaca açıklayınız.

Soru 7 (15 puan)

4 • Değerlendirme Hedef 4

`IntBagliListe::ekleBasa(int)` her çağrıldığında verilen değeri listenin BAŞINA ekler (yeni düğüm kurulur, onun `sonraki`'sine mevcut `bas_` taşınır, sonra yeni düğüm `bas_`'a taşınır). `yazdir() const` ise zinciri baştan sona, her düğümü `deger ->` biçiminde yazıp sonda `nullptr` basarak gezer. Aşağıdaki `main` kesitinin konsola bastığı çıktıyı yazınız:

```
araclar::IntBagliListe liste;
liste.ekleBasa(3);
liste.ekleBasa(2);
liste.ekleBasa(1);
liste.yazdir();
```

Soru 8 (10 puan)

4 • Değerlendirme Hedef 1

`IntTampon` örneği üzerinden, derleyicinin ürettiği varsayılan (sığ) kopya kurucunun neden tehlikeli olduğunu ve kullanıcı tanımlı bir kopya kurucunun (derin kopya ile) bu sorunu nasıl çözdüğünü açıklayınız.

Soru 9 (10 puan)

4 • Değerlendirme Hedef 3

`unique_ptr` ile `shared_ptr`'ı hangi durumlarda tercih edersiniz? **Rule of 3** nedir ve akıllı işaretçiler bu kuralın getirdiği sorumluluğu nasıl hafifletir? Açıklayınız.

Soru 9 (devamı)